

ISCM Phase 10

Typed Pipeline Migration Guide

Migrating Legacy Security Checks to Evidence-Based Typed Evaluation

Document Type	Technical Architecture & Migration Guide
Project	Information Security Compliance Manager (ISCM)
Phase	Phase 10 — Scanner Execution & Typed Pipeline
Branch	epic/phase10-scanner-execution
Status	Complete — All 84 tests passing
Date	September 2026
Audience	Software Architects, Security Engineers, Maintainers

This document describes the complete migration of legacy hardening checks to a modern, typed, evidence-based evaluation pipeline. It serves as both an architectural reference and a practical guide for future extensions.

1. Executive Summary

The Information Security Compliance Manager (ISCM) is an enterprise-grade Windows hardening assessment engine. It evaluates systems against a catalog of security controls and produces evidence-backed findings for compliance reporting.

1.1 Phase 10 Objective

Phase 10 focused on migrating legacy security checks from an embedded-evaluation model to a clean **collector-only** pattern. Evaluation logic was centralized, typed evidence values were introduced, and the scanner became the single source of truth for Pass/Fail decisions.

1.2 Key Achievements

Metric	Result
Legacy checks migrated	18 checks (87 of 114 SubControls)
Coverage of baseline	76.3% of SubControls on typed pipeline
Automated tests	84 tests — all passing
Breaking changes	Zero
Evaluation model	Fully centralized via TypedEvidenceEvaluator
Type safety	EvidenceValue (Boolean, Integer, Duration, Collection, ...)
Graceful degradation	Unknown / non-admin → Fail (never Error)

1.3 Migration Timeline

Sub-Phase	Focus	Tests
10.1	Base infrastructure & contracts	48
10.2	Catalog contract tests	6
10.3	Collector contract tests	5
10.4	Scanner integration	2
10.5	Pilot migration (PWD-001)	3
10.6	Account checks (LCK, GUEST)	2
10.7	Audit checks (AUD, PRC, PSH, EVL)	5
10.8	System / Network batches	10
10.9	CMD + User Rights + Documentation	3
Total		84

1.4 Return on Investment

- **Maintainability** — Evaluation rules live in one place (ControlCatalog + TypedEvidenceEvaluator).
- **Testability** — Collectors can be unit-tested without full system state; evaluation is pure.
- **Type Safety** — No more stringly-typed comparisons; EvidenceValue carries explicit types.
- **Extensibility** — New checks only implement CollectEvidenceAsync; the pipeline does the rest.
- **Consistency** — Identical evaluation semantics across all SubControls.

2. Architecture Overview

2.1 Legacy Architecture (Before Migration)

Prior to Phase 10, each check implemented both data collection and evaluation. The primary interfaces were `IHardeningCheck` and `IMultiPathCheck`.

Problems with the legacy design:

- Evaluation logic was embedded inside every check → tight coupling and duplication.
- No type safety — comparisons were performed on raw strings or objects.
- Difficult to unit-test — many checks required live system state or admin privileges.
- Inconsistent evaluation criteria across similar controls.
- Multi-path verification (`RunMultipleTestsAsync`) further complicated the surface area.

Legacy pattern example:

```
public class PasswordLengthCheck : IHardeningCheck, IMultiPathCheck
{
    public async Task<Finding> EvaluateAsync()
    {
        // Collection + Evaluation mixed together
        var value = ReadRegistry(...);
        if (value < 14)
            return Finding.Fail("Password length too short");
        return Finding.Pass();
    }

    public async Task<List<TestResult>> RunMultipleTestsAsync()
    {
        // Multi-path verification embedded in the check
    }
}
```

2.2 New Typed Pipeline Architecture

The new architecture strictly separates **evidence collection** from **evaluation**. Checks become pure collectors; the Scanner + TypedEvidenceEvaluator become the single evaluation engine.

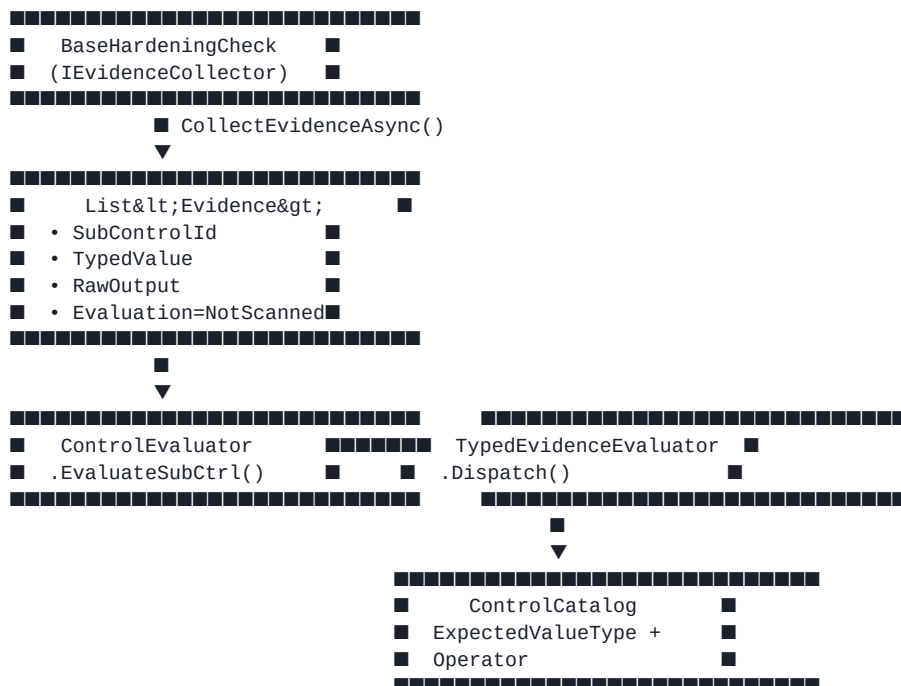


Figure 1 — Typed Pipeline data flow

2.3 Comparison: Before vs After

Aspect	Legacy	Typed Pipeline
Primary interface	IHardeningCheck + IMultiPathCheck	BaseHardeningCheck + IEvidenceCollector
Main method	EvaluateAsync() → Finding	CollectEvidenceAsync() → List<Evidence>
Evaluation location	Inside each check	Centralized in TypedEvidenceEvaluator
Value representation	Untyped / string	EvidenceValue (Boolean, Integer, ...)
Verification	RunMultipleTestsAsync()	Scanner + ControlEvaluator
Error handling	Often Error status	Conservative Fail (never Error)

3. Migration Pattern

Every migrated check follows a consistent template. The check inherits from `BaseHardeningCheck`, implements only evidence collection, and leaves evaluation to the pipeline.

3.1 Canonical Template

```
[SupportedOSPlatform("windows")]
public class SomeCheck : BaseHardeningCheck
{
    private readonly IEvidenceParser _registryParser;

    public override string CheckId => "XXX-001";
    public override string Name => "...";
    public override CheckCategory Category => CheckCategory.XXX;
    public override CheckSeverity Severity => CheckSeverity.XXX;

    public SomeCheck()
    {
        _registryParser = new RegistryParser();
    }

    public override Task<List<Evidence>> CollectEvidenceAsync()
    {
        var evidenceList = new List<Evidence>();
        // One Evidence item per SubControl
        evidenceList.Add(CollectBooleanEvidence("XXX-001.1", regPath, "ValueName", 1));
        return Task.FromResult(evidenceList);
    }

    private Evidence CollectBooleanEvidence(
        string subControlId, string regPath, string valueName, int expected)
    {
        var startTime = DateTime.UtcNow;
        try
        {
            string rawOutput;
            using (var key = Registry.LocalMachine.OpenSubKey(regPath))
            {
                var value = key?.GetValue(valueName);
                rawOutput = value?.ToString() ?? "0";
            }
            var parsedValue = _registryParser.Parse(rawOutput, "Registry");
            var typedValue = ExtractBoolean(rawOutput, expected);

            return new Evidence
            {
                EvidenceId = $"{CheckId}-{subControlId}",
                SubControlId = subControlId,
                SourceType = EvidenceSourceType.Registry,
                SourceName = valueName,
                AcquisitionCommand = $"reg query HKLM\\{regPath} /v {valueName}",
                RawOutput = rawOutput,
                ParsedValue = parsedValue,
                TypedValue = typedValue,
                Evaluation = CheckStatus.NotScanned, // Critical
                CollectedAtUtc = DateTime.UtcNow,
                CollectionDurationMs = (int)(DateTime.UtcNow - startTime).TotalMilliseconds
            };
        }
        catch (Exception ex)
        {
            return CreateFallbackEvidence(subControlId, valueName, ex);
        }
    }
}
```

3.2 Key Rules

1. **One Evidence per SubControl** — Each SubControlId receives exactly one Evidence item.
2. **TypedValue is never null** — Always supply a conservative fallback (treat unknown as insecure).
3. **Evaluation = NotScanned** — The collector never decides Pass/Fail; the Scanner does.
4. **Graceful degradation** — SecurityException or missing data → conservative Fail, never Error status.
5. **Catalog drives evaluation** — ExpectedValueType and Operator are defined in ControlCatalog.cs.

4. EvidenceValue Types

The typed pipeline relies on a discriminated EvidenceValue that carries both the runtime value and its declared type. This eliminates stringly-typed comparisons.

Type	Typical Usage	Construction Example
Boolean	Registry DWORD 0/1	EvidenceValue.FromBoolean(true)
Integer	Numeric DWORD	EvidenceValue.FromInteger(5)
Long	Large integers	EvidenceValue.FromLong(20971520)
String	URLs, KB IDs, names	EvidenceValue.FromString("http://wsus")
Duration	Time (ms → seconds)	new EvidenceValue(new DurationValue(900, DurationUnit.Seconds))
Collection	Audit policies, User rights	new EvidenceValue(list, EvidenceValueType.Collection, rawString: "...")
Enum	Descriptive states	Handled via Catalog ExpectedValueType.Enum

Important Notes

- DurationUnit supports only Seconds, Minutes, Hours, Days — there is no Milliseconds member. Convert milliseconds before construction.
- EvidenceValue.FromCollection() does not exist; use the constructor that accepts a list and EvidenceValueType.Collection.
- ExpectedValueParser.ParseBoolean supports prefix matching (e.g. "Enabled - All drives" → true).

5. Migrated Checks Inventory

The following tables list every check migrated in Phase 10 together with the SubControls they own.

5.1 Phase 10.5 — Pilot (Password Policy)

Check	SubControls
PWD-001 (Password Policy)	PWD-001.1 → PWD-001.6 (6)

5.2 Phase 10.6 — Account Checks

Check	SubControls
LCK-001 (Account Lockout)	LCK-001.1 → LCK-001.3 (3)
GUEST-001 (Guest Account)	GUEST-001.1, GUEST-001.2 (2)

5.3 Phase 10.7 — Audit Checks

Check	SubControls
AUD-001 (Advanced Audit)	AUD-001.1 → AUD-001.11 (11)

Check	SubControls
PRC-001 (Process Creation)	PRC-001.1 → PRC-001.3 (3)
PSH-001 (PowerShell Logging)	PSH-001.1 → PSH-001.3 (3)
EVL-001 (Event Log Size)	EVL-001.1 → EVL-001.6 (6)

5.4 Phase 10.8 — System & Network Checks

Check	SubControls
ARD-001 (Autorun / Autoplay)	ARD-001.1 → ARD-001.3 (3)
UAC-001 (User Account Control)	UAC-001.1 → UAC-001.3 (3)
LM-001 (NTLM / LAN Manager)	LM-001.1, LM-001.2 (2)
CRG-001 (Credential Guard)	CRG-001.1 → CRG-001.6 (6)
FW-001 (Firewall)	FW-001.1 → FW-001.4 (4)
LLN-001 (LLMNR / NetBIOS)	LLN-001.1 → LLN-001.7 (7)
SMB-001 (SMBv1)	SMB-001.1 → SMB-001.5 (5)
WUP-001 (Windows Update)	WUP-001.1 → WUP-001.4 (4)
RDP-001 (Secure RDP)	RDP-001.1 → RDP-001.7 (7)

5.5 Phase 10.9 — CMD & User Rights

Check	SubControls
CMD-001 (Disable CMD)	CMD-001.1 → CMD-001.3 (3)
URA-001 (User Rights)	URA-001.1 → URA-001.9 (9)

Total migrated: 18 checks covering 87 SubControls.

6. Catalog Alignment Changes

During migration a number of SubControls required updates to their ExpectedValueType or Operator so that the TypedEvidenceEvaluator could evaluate them correctly.

SubControl	Before	After
WUP-001.1	Boolean	Integer (GreaterOrEqual 4)
WUP-001.3	Enum	Integer (Equals 1)
WUP-001.4	Boolean	String (Contains "KB")
RDP-001.2	Boolean	Integer (GreaterOrEqual 3)
RDP-001.5	Integer	Integer (LessOrEqual 5)
RDP-001.7	Duration	Integer (LessOrEqual 60000)

Known remaining mismatches (deferred to Phase 11)

- EVL-001.5 / EVL-001.6 — Catalog defines Enum; check currently produces Integer/Boolean.
- FW-001.2 / FW-001.3 — Catalog expects Enum string; check produces Boolean.

These produce Fail (not Error) and are scheduled for cleanup in the next phase.

7. Running the Test Suite

7.1 Commands

```
# Full suite
dotnet test ISCM.Tests\ISCM.Tests.csproj

# Targeted groups
dotnet test --filter "FullyQualifiedName~AuditChecksRealSystemTests" --logger "console;verbosity=detailed"
dotnet test --filter "FullyQualifiedName~RegistryChecksBatch1RealSystemTests" --logger "console;verbosity=detailed"
dotnet test --filter "FullyQualifiedName~NetworkChecksBatch2RealSystemTests" --logger "console;verbosity=detailed"
dotnet test --filter "FullyQualifiedName~SecureRdpCheckRealSystemTests" --logger "console;verbosity=detailed"
dotnet test --filter "FullyQualifiedName~CmdAndUserRightsChecksRealSystemTests" --logger "console;verbosity=detailed"
```

7.2 Expected Behaviour on Non-Admin Machines

Check Group	Observed Behaviour
auditpol-based (AUD-001, PRC-001)	TypedValue empty → all Fail (requires admin)
Get-WindowsOptionalFeature (SMB-001.1)	TypedValue "" → Fail (requires admin)
secedit (URA-001)	TypedValue empty → all Fail (requires admin)
Security log registry (EVL-001.2)	TypedValue=0 → Fail (graceful degradation)
All remaining checks	Normal Pass/Fail according to system state

Key principle: No SubControl may return Error status. Unknown or inaccessible data is treated as Fail (conservative / insecure).

8. Deferred Items & Future Work

8.1 Intentionally Not Migrated

Check	Reason
ADM-001 (Admin Account Count)	WMI Win32_GroupUser complexity; low baseline priority
DEF-001 (Windows Defender)	Extended check — outside current baseline
USB-001 (USB Storage Policy)	Extended check — outside current baseline
ALG-001 (AutoLogon)	Extended check — outside current baseline

8.2 Phase 11 Candidates

- Complete migration of ADM-001 using a robust WMI collector.
- Bring DEF-001, USB-001 and ALG-001 onto the typed pipeline if they enter the baseline.
- Resolve the remaining catalog mismatches for EVL-001.5/6 and FW-001.2/3.
- Consider introducing a Duration unit conversion helper to reduce boilerplate.

9. Test Coverage Summary

Phase	Focus Area	Tests	Status
10.1	Base tests	48	Pass
10.2	Catalog contract	6	Pass
10.3	Collector contract	5	Pass
10.4	Scanner integration	2	Pass
10.5	Pilot migration	3	Pass
10.6	Account checks	2	Pass
10.7	Audit checks	5	Pass
10.8 Batch 1	Registry	4	Pass
10.8 Batch 2	Network	4	Pass
10.8 Batch 3	RDP	2	Pass
10.9 Batch 4	CMD + URA	3	Pass
Total		84	All Pass

10. Conclusion

Phase 10 successfully transformed the ISCM evaluation engine from a tightly-coupled, stringly-typed design into a clean, typed, evidence-driven pipeline. Eighteen checks and eighty-seven SubControls now collect evidence only; evaluation is centralized, consistent and fully covered by automated tests.

Future developers can add new checks by implementing a single method (`CollectEvidenceAsync`) and declaring the expected type and operator in the `ControlCatalog`. The rest of the pipeline remains unchanged.

Branch: epic/phase10-scanner-execution

Completion: September 2026

Status: Closed — ready for Phase 11 planning.